

Turbos

Audit Report

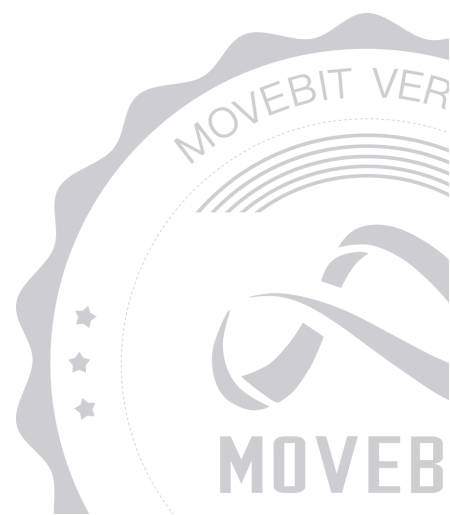


contact@bitslab.xyz



https://twitter.com/movebit_

Fri Nov 28 2025



Turbos Audit Report

1 Executive Summary

1.1 Project Information

Description	Turbos is a DeFi protocol that integrates staking and swap functionalities. It allows users to deposit assets into the Bank module, which mints corresponding bTokens based on the user's share of the current asset pool. The assets staked in the Bank are then utilized in the DeepBook Margin lending protocol, generating passive income for users. Meanwhile, the minted bTokens can be used in the protocol's swap feature. The swap module of Turbos follows the classic design principles of Uniswap V2. In its underlying trading pools, users can mint liquidity tokens proportionally according to the current pool reserves. Unlike Uniswap V2, however, Turbos requires liquidity tokens to be separately staked in the Rewards module to earn yields. Turbos is designed to support multiple pricing models. In the current code version, it implements two distinct market-making models—CPMM and OMM—each employing different pricing mechanisms. The CPMM adopts the classic $X*Y=K$ formula, while OMM calculates exchange rates by synthesizing external oracle prices and the current liquidity ratio of the trading pool, effectively reducing slippage.
Type	Dex

Auditors	Bear Two, s3cunda, N0n3
Timeline	Tue Nov 04 2025 - Fri Nov 28 2025
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/turbos-finance/turbos-omm-contract
Commits	3ee82f23c1c9a70a30c98018d7e8b235a0f0e0d0d2901a7f56c3f9fdbf577a5dbf1f342caf1ec13f781f8668286e57c71e0a302d7dc43a620293b659bf19afefcb9dc23dbd3fbb068f4c59361508f40

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
MTE	contracts/btokens/b_dbusdc/Move.testnet.toml	5a02405cf0dcd709b22912add1aafb6c31326d30
MOV	contracts/btokens/b_dbusdc/Move.toml	191c194ac5b9cb1d0d29ed9e98f82b6be106a081
BDB	contracts/btokens/b_dbusdc/sources/b_dbusdc.move	d52d731cfb913effcfb65500a619764a84dea85b
MTE1	contracts/btokens/b_sui/Move.testnet.toml	d338a317577856ebbcf3fe4a3e3357bbeec951f4
MOV1	contracts/btokens/b_sui/Move.toml	1e4cd2815b6e71f8c59c6ba8ef4f6b79aad64f1e
BSU	contracts/btokens/b_sui/sources/b_sui.move	e915865731bb85c4f991aeb138ea3655aae698a
MTE2	contracts/turbos_omm/Move.testnet.toml	3831c976ba0526d66e5daf318b943b0ca92a57ff
ACL	contracts/turbos_omm/sources/acle.move	94d37546fc7f09ce84f66195a9f0dc9cf32b5cb5
ODE	contracts/turbos_omm/sources/oracle/oracle_decimal.move	f5ec60e78133d0a699ff7be0e94c34f6e045f6b2
ORA	contracts/turbos_omm/sources/oracle/oracles.move	2ab7a39c945379fffbcb9e827c32bf19b2eb3aac

PYT	contracts/turbos_omm/sources/oracle/pyth.move	4a27e9f3f5e479e1f0762c7382e46a1527707fff
GCO	contracts/turbos_omm/sources/global_config.move	d8efd359a9ec76e20d05aba23938ef8018f4bdb3
FEE	contracts/turbos_omm/sources/fees.move	5bf5b5d40aa716e23992790629b217863b461957
POO	contracts/turbos_omm/sources/pool/pool.move	6d4fd784ef5358d9074848848cb9da3329e75611
PMA	contracts/turbos_omm/sources/pool/pool_math.move	c0d48784b808aa5ffec0ad451b289355bc62f6a6
REW	contracts/turbos_omm/sources/pool/rewards.move	4ce782a24267336f5cbbd22feec4fec7603c9af3
FCR	contracts/turbos_omm/sources/fee_crank.move	e2c0ab7dd77138d734c75f840c37451179d715c0
QUO	contracts/turbos_omm/sources/quote.move	e4e37c53cb48c01b7d29359e3ab09c4ef6ed0dd3
UTI	contracts/turbos_omm/sources/utis.move	0079171769eb713be7c8747ea38d117d42d4d174
BMA	contracts/turbos_omm/sources/bank/bank_math.move	7ebbfd3ea0d53a674ea9abc107e00854b9e3259
BAN	contracts/turbos_omm/sources/bank/bank.move	db465f65a117376eb41aa7b0a2bc baf1c1fb8cb7
M12	contracts/turbos_omm/sources/math/math128.move	b8b23f11f24f6b39d4714f2c5c9b3421b30f74c1

DEC	contracts/turbos_omm/sources/math/decimal.move	85da15a6273c72eb8f41dec5e4410c5ff17670bb
FP6	contracts/turbos_omm/sources/math/fixed_point64.move	e4ae675b1d5b2fac880546d7afa05e0373df63e1
BMA1	contracts/turbos_omm/sources/math/base_math.move	0015c3a4ff3ffd1dfa7d41a100c623df486905f4
M25	contracts/turbos_omm/sources/math/math256.move	0be3ab5cf84e3706bb671b6a23bfd9ceb2b218ac
REG	contracts/turbos_omm/sources/registry.move	06de16a47ff59868e3fbe164491c26205eaae105
CPM	contracts/turbos_omm/sources/quoters/cpmm.move	c585023a4fda74837babdc74263c434daa1a1b84
OMM	contracts/turbos_omm/sources/quoters/omm.move	06666d97f6a3ab8353d6e63c731bcee083f652da
MTE3	contracts/lptokens/turbos_lp_bsui_bdbusdc/Move.testnet.toml	16635fbfd1f3455a6f221b89054b4413be580037
MOV2	contracts/lptokens/turbos_lp_bsui_bdbusdc/Move.toml	382aa3b294e16668994997053fbefc619d1dccf
TLBD	contracts/lptokens/turbos_lp_bsui_bdbusdc/sources/turbos_lp_bsui_bdbusdc.move	02d0285e6bfc9028956cb0276405e8953ca72927
MTE4	contracts/turbos_omm_scripts/Move.testnet.toml	4e8302f7c02952b1b5d71d724640723dfb880081
BSC	contracts/turbos_omm_scripts/sources/bank_scripts.move	a18e2e0dd6a41496da7da6db4ec4e482807ce157

PSC	contracts/turbos_omm_scripts/sources/pool_scripts.move	f3b109b2b7c2f7be527468a8c3c069a4b30e2f19
-----	--	--

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	17	14	3
Critical	0	0	0
Major	2	2	0
Medium	7	6	1
Minor	4	4	0
Informational	4	2	2

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the "**Testing and Automated Analysis**", "**Code Review**" and "**Formal Verification**" strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by Turbos to identify any potential issues and vulnerabilities in the source code of the Turbos smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 17 issues of varying severity, listed below.

ID	Title	Severity	Status
BAN-4	Inappropriate Calculation in Determining Destroyed Shares During <code>withdraw</code> Execution	Major	Fixed
OMM-1	A Discrepancy Between Liquidity Ratio and Oracle Price May Cause Swap Function Failure	Major	Fixed
PMA-3	Asymmetric Liquidity Can Freeze the Pool	Medium	Fixed
POO-6	Error in View Function <code>redeem_result_withdraw_b</code>	Minor	Fixed
POO-7	Duplicate Error Conditions Share the Same Error Code	Informational	Fixed
REW-5	Overly Restrictive Access Control on <code>fund</code>	Minor	Fixed
BAN-17	The Absence of a Non-Zero Return Value Check in the <code>to_btokens</code>	Medium	Fixed

	Function Resulted in Fund Losses		
BAN-19	Lending Feature Cannot Be Disabled, Posing Potential Financial Risks	Minor	Fixed
BAN-20	Potential Failure of the <code>claim_fees</code> Fee Extraction Routine Under Specific Conditions	Medium	Fixed
BAN-22	Redundant and Inconsistent Balance Checks in <code>burn_btoken</code> Withdraw Logic	Informational	Fixed
OMM-12	Returning 0 When Swap Output Depletes Reserve Pool Asset Loss	Medium	Fixed
OMM-13	Missing Validation on <code>amplifier</code> When Create New Pool	Minor	Fixed
OMM-16	Potential Arbitrage Opportunities	Informational	Acknowledged
POO-18	Issues Regarding Liquidity Token Minting, Burning, and Staking Rewards	Informational	Acknowledged
PSC-11	Redundant Slippage Checks May Cause Unnecessary Transaction Failures	Medium	Fixed
REW-23	Missing Check Between <code>start_ts</code> and <code>clock.timestamp_ms()</code>	Medium	Fixed
REW-24	Inconsistent State Updates During Reward Injection May Lead to	Medium	Acknowledged

	Incorrect User Earnings Calculations		
--	---	--	--

3 Participant Process

Here are the relevant actors with their respective abilities within the [Turbos](#) Smart Contract :

Admin

bank:

- `create_bank_and_share` & `create_bank` - Create and register bank object.
- `init_lending` - Initialize bank module's lending parameters.
- `set_utilisation_bps` - Set `target_utilisation_bps` and `utilisation_buffer_bps` in bank module.
- `set_min_token_block_size` - Set `min_token_block_size` in bank module.
- `set_referral` - Set `referral` in bank lending module.

reward:

- `register_pool` - One time register a pool for rewards tracking.
- `add_reward_token` - Add a reward token for a pool.
- `schedule_reward` - Configure time window and emission with budget safety.
- `fund` - Deposit reward tokens into a pool-specific reward vault.
- `withdraw` - Withdraw all withdrawable funds from rewards pool.

cpmm & omm:

- `new` - Creates a new constant product AMM & oracle pricing pool.
- `swap` - Swap btoken by using constant product & oracle pricing.
- `pause_pool` - Pause pool by setting `is_paused` in `global_config` .
- `resume_pool` - Resume pool by setting `is_paused` in `global_config` .

User

bank:

- `mint_btokens` & `mint_btoken` - Mint btokens by transfer corresponding underlying token into bank module.

- burn_btokens & burn_btoken - Burn btoken and redeem corresponding underlying token.

reward:

- stake - Stake LP tokens into the rewards module.
- unstake - Unstake LP tokens.
- claim - Claim rewards for a specific reward token.

4 Findings

BAN-4 Inappropriate Calculation in Determining Destroyed Shares During `withdraw` Execution

Severity: Major

Status: Fixed

Code Location:

`contracts/turbos_omm/sources/bank/bank.move#698`

Descriptions:

In the `withdraw` function, `shares_burned` is calculated as `user_shares_after - user_shares`. However, `user_shares` is retrieved before calling `lending_market.withdraw`, while `user_shares_after` is retrieved after the call. The `lending_market.withdraw.withdraw` function [reduces](#) the value of `user_supply_shares`.

Therefore, when calculating `shares_burned`, the value of `user_shares_after` will be less than `user_shares`, causing the transaction to roll back. This will result in `rebalance` and `withdraw` failing.

Suggestion:

It is recommended to change the order of subtraction operations to calculate the destroyed shares correctly.

Resolution:

The team adopted our advice and fixed this issue by changing the order of subtraction operations to calculate the destroyed shares correctly, which can be found at `667dc7445cad335b122ca873ffc2064e2c94d00`.

OMM-1 A Discrepancy Between Liquidity Ratio and Oracle Price May Cause Swap Function Failure

Severity: Major

Status: Fixed

Code Location:

contracts/turbos_omm/sources/quoters/omm.move

Descriptions:

In the `OMM` protocol, token swap calculations take into account both external oracle prices and the current liquidity in the pool. Compared to direct reliance on constant price formula-based swap methods, this approach effectively mitigates the occurrence of slippage.

During the exchange process, `OMM` first computes the swap quote (by calling the `quote_swap_` function). In this function, it initially calculates the outcome based on a comprehensive consideration of external oracle prices and current pool liquidity. This result is then compared with a slippage-free exchange outcome (i.e., the result derived directly from external oracle prices). If the outcome from the `OMM` method is greater than or equal to the slippage-free result, the transaction will fail.

contracts/turbos_omm/sources/quoters/omm.move

```
fun quote_swap_<B_A, B_B, LpType: drop>(
  pool: &Pool<B_A, B_B, OracleQuoter, LpType>,
  global_config: &GlobalConfig,
  price_a: Decimal,
  price_b: Decimal,
  price_uncertainty_ratio_a: u64,
  price_uncertainty_ratio_b: u64,
  btoken_ratio_a: Decimal,
  btoken_ratio_b: Decimal,
  underlying_reserve_a: Decimal,
  underlying_reserve_b: Decimal,
  amount_in: u64,
```

```

a2b: bool,
): SwapQuote {
    let quoter = pool.quoter();
    let decimals_a = quoter.decimals_a;
    let decimals_b = quoter.decimals_b;

    let amount_out = if (a2b) {
        let underlying_amount_in = decimal::from(amount_in).mul(btoken_ratio_a);
        let underlying_reserve_in = underlying_reserve_a;
        let underlying_reserve_out = underlying_reserve_b;

        let amount_out_underlying = get_swap_output(
            underlying_amount_in,
            underlying_reserve_in,
            underlying_reserve_out,
            price_a,
            price_b,
            decimals_a as u64,
            decimals_b as u64,
            quoter.amp,
            true,
        );

        let mut amount_out =
decimal::from(amount_out_underlying).div(btoken_ratio_b).floor();

        if (amount_out >= pool.balance_amount_b()) {
            amount_out = 0
        };

        amount_out
    } else {
        let underlying_amount_in = decimal::from(amount_in).mul(btoken_ratio_b);
        let underlying_reserve_in = underlying_reserve_b;
        let underlying_reserve_out = underlying_reserve_a;

        let amount_out_underlying = get_swap_output(
            underlying_amount_in,
            underlying_reserve_out,

```

```

    underlying_reserve_in,
    price_a,
    price_b,
    decimals_a as u64,
    decimals_b as u64,
    quoter.amp,
    false,
);

let mut amount_out =
decimal::from(amount_out_underlying).div(btoken_ratio_a).floor();

if (amount_out >= pool.balance_amount_a()) {
    amount_out = 0;
};

amount_out
};

let amount_out_with_no_slippage = if (a2b) {
    quote_swap_impl(
        amount_in,
        decimals_a,
        decimals_b,
        price_a,
        price_b,
        btoken_ratio_a,
        btoken_ratio_b,
    )
} else {
    quote_swap_impl(
        amount_in,
        decimals_b,
        decimals_a,
        price_b,
        price_a,
        btoken_ratio_b,
        btoken_ratio_a,
    )
}

```

```
};

assert!(amount_out < amount_out_with_no_slippage, EInvalidPrice);

pool.get_quote(
    global_config,
    amount_in,
    amount_out,
    a2b,
    some(price_uncertainty_ratio_a.max(price_uncertainty_ratio_b)),
)
}
```

In terms of design, the above solution is based on the premise that under ideal conditions - specifically when the liquidity ratio in the pool closely aligns with the external oracle price ratio - the **OMM** formula doesn't completely eliminate slippage but rather reduces it to some extent. Therefore, the **OMM** method's result should always be inferior to the slippage-free outcome. However, this assumption relies on the condition that the pool's liquidity ratio remains relatively consistent with the market price ratio, which may not hold true in all circumstances.

For example, consider a pool containing two tokens, A and B, with a market price ratio of 1:1. Ideally, when the pool's liquidity ratio maintains at 1:1, the assumption holds true - the **OMM** method's result will indeed be worse than the slippage-free outcome. However, if the pool's liquidity ratio shifts from 1:1 to 1:2, then when swapping A for B, the **OMM** calculation would yield a result better than the slippage-free outcome. This would cause the swap functionality to malfunction: any attempt to exchange A for B would be rejected, while only B-to-A swaps would execute successfully.

Suggestion:

To address this issue, we recommend against using a unilateral greater-than or less-than check as the determining condition. Instead, we propose establishing an acceptable range for exchange outcomes. Transactions should be allowed to proceed as long as the deviation

between the OMM result and the slippage-free outcome falls within an acceptable tolerance range.

Furthermore, to prevent scenarios where the pool's liquidity ratio significantly diverges from the external oracle price, we recommend that projects implement adjustment mechanisms. These should allow users to withdraw their assets when such depegging situations occur.

Resolution:

The team adopted our advice and fixed this issue by removing the oracle price(zero-slippage) check, which can be found at [f781f8668286e57c71e0a302d7dc43a620293b65](https://github.com/Uniswap/v2-core/blob/master/contracts/UniswapV2OraclePriceCheck.sol).

PMA-3 Asymmetric Liquidity Can Freeze the Pool

Severity: Medium

Status: Fixed

Code Location:

contracts/turbos_omm/sources/pool/pool_math.move#92-109

Descriptions:

In the `tokens_to_deposit` function, the logic for adding liquidity behaves asymmetrically when one of the pool's token reserves is zero. When `reserve_b == 0` while `reserve_a > 0`, users are allowed to successfully add liquidity using only token A. However, when `reserve_a == 0` while `reserve_b > 0`, any attempt to add liquidity fails due to `assert!(a_star > 0, EDepositRatioLeadsToZeroA)`, because the computation path in this scenario always results in `a_star = 0`.

This asymmetrical behavior is a logical flaw that unfairly treats the two tokens in the pair differently. More importantly, if `reserve_a` is accidentally depleted, the pool may become permanently unable to accept new liquidity, which can harm the protocol's robustness.

Suggestion:

It is recommended to refactor the logic in `tokens_to_deposit` so that it handles the zero-reserve case symmetrically. When `reserve_a` is zero, users should be allowed to deposit token B.

Resolution:

The team adopted our advice and fixed this issue by removing the checks that `max_a` and `a_star` are greater than 0, allowing only the b token to be added when `reserve_a` is 0, which can be found at 9bf19afefcb9dc23dbd3fbb068f4c59361508f40.

POO-6 Error in View Function `redeem_result_withdraw_b`

Severity: Minor

Status: Fixed

Code Location:

`contracts/turbos_omm/sources/pool/pool.move#408`

Descriptions:

The function `redeem_result_withdraw_b` is intended to return the `withdraw_b` field of the `RedeemResult` structure, which represents the amount of tokenB withdrawn. However, the function currently returns `redeem_result.withdraw_a` instead.

This causes off-chain applications or user interfaces that rely on this view function to receive incorrect token B values, potentially leading to user confusion or incorrect UI displays.

Suggestion:

It is recommended to adjust the withdrawal-result logic so that the function returns the correct `B-asset` withdrawal value instead of incorrectly returning the `A-asset` value, ensuring the output matches the intended asset type.

Resolution:

The team adopted our advice and fixed this issue by changing `redeem_result.withdraw_a` of `redeem_result_withdraw_b` function to `redeem_result.withdraw_b`, which can be found at `f781f8668286e57c71e0a302d7dc43a620293b65`.

POO-7 Duplicate Error Conditions Share the Same Error Code

Severity: Informational

Status: Fixed

Code Location:

contracts/turbos_omm/sources/pool/pool.move#44,45

Descriptions:

In the contract's constant definitions, the errors `ELpTokenEmpty` and `EEmptyCoins` are both assigned the same error code `9`. As a result, when `redeem_liquidity` fails due to the `LP` token amount being zero, or when `deposit_liquidity` fails because both input token amounts are zero, the transaction aborts with the same error code `9`.

This makes it difficult for off-chain tools and developers to distinguish the exact cause of failure, increasing the complexity of debugging and reducing clarity in error handling.

Suggestion:

It is recommended to assign a unique error code to each distinct error condition.

Resolution:

The team adopted our advice and fixed this issue by changing the `EmptyCoins` error code to `10`, which can be found at `f781f8668286e57c71e0a302d7dc43a620293b65`.

REW-5 Overly Restrictive Access Control on `fund`

Severity: Minor

Status: Fixed

Code Location:

contracts/turbos_omm/sources/pool/rewards.move#293

Descriptions:

The `fund` function is responsible for depositing reward tokens into the pool's reward vault. It performs an access control check using:

```
config.check_reward_manager_role(ctx.sender())
```

This means that only the designated administrator role is authorized to supply funds to the rewards pool. While this does prevent meaningless or malicious token deposits, it also introduces a significant centralization risk.

If the administrator account becomes lost, inactive, or malicious, the entire protocol's reward mechanism can be disrupted, as no new reward funds can be added. Such reliance on a centralized authority for a core operational function may undermine user trust and the long-term sustainability of the protocol.

Suggestion:

It is recommended to reevaluate the access control strategy for the `fund` function. Consider removing the administrator-only restriction and allowing anyone to contribute funds to the reward pool.

Resolution:

The team adopted our advice and fixed this issue by removing the rewards administrator check, which can be found at `f781f8668286e57c71e0a302d7dc43a620293b65`.

BAN-17 The Absence of a Non-Zero Return Value Check in the `to_btokens` Function Resulted in Fund Losses

Severity: Medium

Status: Fixed

Code Location:

`contracts/turbos_omm/sources/bank/bank.move#345`

Descriptions:

The `to_btokens` function computes the number of bTokens to mint for a given deposit amount using the `ratio amount * btoken_supply / total_funds` and applies `Decimal::floor()`. When the deposit amount is very small relative to the total pool size, this calculation can round down to zero. In `mint_btoken / mint_btokens`, the returned value from `to_btokens` is used directly without checking that it is greater than zero. As a result, a user can deposit a non zero amount of underlying tokens, increasing `funds_available`, while receiving 0 bTokens in return. This creates a clear economic loss for the depositor and allows value to be silently transferred to existing bToken holders.

Suggestion:

Add an explicit check after calling `to_btokens` in `mint_btoken` to ensure the returned `new_btokens` is strictly greater than zero.

Resolution:

The team adopted our advice and fixed this issue by adding a check that the calculated `new_btokens` is greater than 0, which can be found at `f781f8668286e57c71e0a302d7dc43a620293b65`.

BAN-19 Lending Feature Cannot Be Disabled, Posing Potential Financial Risks

Severity: Minor

Status: Fixed

Code Location:

contracts/turbos_omm/sources/bank/bank.move

Descriptions:

The state machine of the `Bank` contract is incomplete. The `init_lending` function can transition the `lending` state from `None` to `Some` to activate lending functionality, but the contract lacks a mechanism to transition the state back from `Some` to `None`. This means that once lending is activated, it cannot be disabled. If the externally dependent `MarginPool` (lending market) experiences issues—such as attacks, vulnerabilities, or protocol upgrades—bank administrators cannot decouple the `Bank` from it. This causes functions like `rebalance` to persistently interact with the problematic market, potentially exposing user funds to risk or causing transaction failures.

Suggestion:

It is recommended to add a `disable_lending` function invoked by the bank administrator. This function should first withdraw all deployed funds from the `MarginPool`. This will enable administrators to safely suspend or permanently halt lending functionality when necessary, effectively isolating the bank from external markets and mitigating risk.

Resolution:

The team adopted our advice and fixed this issue by adding the `disable_lending` function, which can be found at `9a8abc2e81c701aca036020635d07744b3fe696b`.

BAN-20 Potential Failure of the `claim_fees` Fee Extraction Routine Under Specific Conditions

Severity: Medium

Status: Fixed

Code Location:

`contracts/turbos_omm/sources/bank/bank.move#446`

Descriptions:

The `claim_fees` function redeems accumulated `BToken` fees for the underlying asset `T` by calling `burn_btoken`. The `burn_btoken` function includes a safeguard to prevent the total supply of `BToken` from falling below `MINIMUM_LIQUIDITY`. If burning fees would cause the supply to drop below this threshold, `burn_btoken` reduces the actual amount burned. This results in the `fee_btoken Coin` object within the `claim_fees` function retaining a balance after the call. However, the function ultimately calls `fee_btoken.destroy_zero()`, which requires the `Coin` value to be zero or else causes the transaction to roll back. Consequently, when the total `BToken` supply approaches the minimum threshold, the fee extraction mechanism becomes blocked.

Suggestion:

After invoking `burn_btoken`, it is recommended to verify whether any value remains in `fee_btoken`. If residual value exists, the contract should handle it appropriately rather than proceeding with the burn operation.

Resolution:

The team adopted our advice and fixed this issue by adding a check mechanism and properly handling the remaining funds, which can be found at `9a8abc2e81c701aca036020635d07744b3fe696b`.

BAN-22 Redundant and Inconsistent Balance Checks in `burn_btoken` Withdraw Logic

Severity: Informational

Status: Fixed

Code Location:

`contracts/turbos_omm/sources/bank/bank.move#265-273`

Descriptions:

The `burn_btoken` function exhibits a pattern of redundant and internally inconsistent logical checks when determining the final token withdrawal amount (`tokens_to_withdraw`). This design choice unnecessarily increases `gas` consumption and detracts from code clarity. The function first enforces a strict precondition using an assertion:

```
assert!(bank.funds_available.value() >= tokens_to_withdraw, ElnsufficientBankFunds);
```

This assertion already guarantees that `tokens_to_withdraw` cannot exceed the total available funds (i.e., `max_available`). Immediately following this strict check, the code introduces a comment and subsequent logic that softens the constraint, allowing for an over-withdrawal of up to 1 unit due to rounding, before explicitly capping the amount: `tokens_to_withdraw = tokens_to_withdraw.min(max_available);` . Since the initial assert ensures that `tokens_to_withdraw ≤ max_available` , the subsequent `min()` operation and its associated condition are redundant; this redundant logic results in unnecessary gas consumption for a check whose validity has already been guaranteed by the preceding code.

Suggestion:

It is recommended to remove redundant parts to simplify this code.

Resolution:

The team adopted our advice and fixed this issue by removing redundant parts, which can be found at `9a8abc2e81c701aca036020635d07744b3fe696b`.

OMM-12 Returning 0 When Swap Output Depletes Reserve Pool Asset Loss

Severity: Medium

Status: Fixed

Code Location:

contracts/turbos_omm/sources/quoters/omm.move

Descriptions:

In the `get_swap_output` function, if the calculated swap output amount (`delta_out`) is greater than or equal to the entire reserve of the target asset, the function returns 0 instead of reverting the transaction. This value of 0 propagates up to the swap function. If users set the `min_amount_out` parameter to 0 when calling the swap function, the transaction will execute successfully. Users will transfer their tokens in but receive 0 output tokens, resulting in a complete loss of their input funds.

contracts/turbos_omm/sources/quoters/omm.move

```
if (x2y) {
    if (delta_out >= reserve_y.floor()) {
        return 0
    };
} else {
    if (delta_out >= reserve_x.floor()) {
        return 0
    };
};
```

Suggestion:

It is recommended to modify the logic of the `get_swap_output` function. When the output amount exceeds the available reserves, the function should use `assert!` to throw an error and revert the transaction, rather than returning 0.

Resolution:

The team adopted our advice and fixed this issue by revert transaction when the output amount exceeds the available reserves, which can be found at [f781f8668286e57c71e0a302d7dc43a620293b65](#).

OMM-13 Missing Validation on `amplifier` When Create New Pool

Severity: Minor

Status: Fixed

Code Location:

`contracts/turbos_omm/sources/quoters/omm.move`

Descriptions:

In the `new` and `new_and_share` functions, the `amplifier` parameter is not validated when creating new pools. However, the `update_amplifier` function correctly restricts it to the range `[1, 10_000]`. If an administrator passes an invalid value (e.g., 0) during pool creation, the pool will be successfully created. But when any user attempts to execute a swap or get a quote (`quote_swap`) in this pool, the transaction will fail due to a division-by-zero error in internal mathematical calculations. This will permanently disable the core functionality of the pool.

Suggestion:

It is recommended to add the same validation logic in the `new` function as in the `update_amplifier` function to ensure the amplifier parameter falls within the valid range at the time of pool creation.

Resolution:

The team adopted our advice and fixed this issue by add validation logic in `new` function, which can be found at `f781f8668286e57c71e0a302d7dc43a620293b65`.

OMM-16 Potential Arbitrage Opportunities

Severity: Informational

Status: Acknowledged

Code Location:

contracts/turbos_omm/sources/quoters/omm.move

Descriptions:

The Turbos protocol incorporates two distinct automated market maker (AMM) mechanisms, namely CPMM and OMM. The key difference between them lies in their pricing methodologies: CPMM employs the constant product formula ($x*y=k$) to determine prices, whereas OMM utilizes a composite formula that integrates oracle price feeds and local liquidity reserves. In the codebase, this distinction is reflected in the Quoter type defined for each trading pool.

Due to this design, the protocol permits the existence of two separate trading pools for the same pair of token assets—one using the CPMM Quoter and the other the OMM Quoter. Given the differing pricing formulas, a price spread between the two pools is inevitable. This raises the question of whether this design is intentional. Since only administrators are authorized to create trading pools, it is essential to clarify whether the protocol intentionally allows two pools with different pricing mechanisms for the same token pair.

Suggestion:

If such a potential arbitrage pathway indeed exists, please clarify whether it was a deliberate design choice or an oversight. If this arbitrage path is not intended, appropriate mitigation measures should be proposed and explained.

Resolution:

The team acknowledged this issue and state that "In its design, Turbos permits two trading pools for the same pair to coexist, each using a distinct price calculation formula. However, the OMM is specifically intended for tokens that have relatively stable prices and substantial

liquidity. As such, in terms of configuration, administrators will apply constraints during the pool setup process.", so no modifications will be made to the current version.

POO-18 Issues Regarding Liquidity Token Minting, Burning, and Staking Rewards

Severity: Informational

Status: Acknowledged

Code Location:

contracts/turbos_omm/sources/pool/pool.move;
contracts/turbos_omm/sources/quoters/omm.move

Descriptions:

In Turbos' design, regardless of whether the CPMM or OMM pricing formula is used, the underlying code logic for pools remains consistent, as do the methods for liquidity provision and liquidity token minting. When liquidity tokens are minted, the minting logic follows the CPMM specification—that is, tokens are minted proportionally based on the current asset liquidity ratio in the trading pool.

However, an issue arises because OMM employs a different pricing formula yet relies on CPMM-based liquidity minting calculations. This inconsistency leads to inaccuracies in liquidity token minting, burning, and staking rewards.

During minting and burning, the current pool price follows the OMM formula, while the minting process uses the CPMM-based asset ratio. As a result, liquidity provided at the current OMM price does not reflect OMM-based proportions, causing users to receive fewer LP tokens than expected. This, in turn, reduces their staking rewards in the reward contract. Similarly, when burning LP tokens, users do not receive assets corresponding to the OMM price ratio.

Suggestion:

It is recommended to clarify the design principles—specifically, whether the discrepancy between the pricing formula and the liquidity minting logic was intentionally considered

during the design phase. If so, an explanation of the design rationale should be provided. If not, we suggest making appropriate modifications to the relevant code.

Resolution:

The team acknowledged this issue and state that "This design is intentional. LP tokens represent a proportional claim on the pool's actual reserves. Minting based on OMM oracle prices (which may deviate from reserve ratios) would break the invariant $\text{Total LP Supply} \leq \text{Total Reserves}$, leading to potential value leakage or dilution. Liquidity provision must adhere to reserve ratios to maintain solvency, while OMM pricing is strictly for optimizing swap execution.", so no modifications will be made to the current version.

PSC-11 Redundant Slippage Checks May Cause Unnecessary Transaction Failures

Severity: Medium

Status: Fixed

Code Location:

contracts/turbos_omm_scripts/sources/pool_scripts.move

Descriptions:

In the `cleanup_swap` function, the slippage check `assert!(coin_x.value() >= min_amount_out, ESlippageExceeded)` is incorrectly applied to all remaining btoken assets (`btoken_a` and `btoken_b`). However, in a transaction, one token is the output token (which should undergo slippage checks), while the other is the remaining input token (which should not be subject to slippage checks). When a small amount of the input token remains after the transaction (i.e., "dust"), applying the slippage check to these residual tokens will cause the assertion to fail, as their value is almost always less than `min_amount_out`. This leads to legitimate transactions unnecessarily rolling back, significantly compromising the reliability of the protocol's core swap functionality.

Suggestion:

It is recommended to determine the type of output token during `cleanup_swap` and apply slippage checks only to the output token.

Resolution:

The team adopted our advice and fixed this issue by determine the type of output token by `a2b` value in `cleanup_swap` and apply slippage checks only to the output token, which can be found at 9bf19afefcb9dc23dbd3fbb068f4c59361508f40.

REW-23 Missing Check Between `start_ts` and `clock.timestamp_ms()`

Severity: Medium

Status: Fixed

Code Location:

`contracts/turbos_omm/sources/pool/rewards.move`

Descriptions:

The `schedule_reward` function in the `rewards` contract is responsible for configuring the reward time window and the reward tokens. However, when setting the time window, there is no validation to check the relationship between the current timestamp and the start timestamp of the reward period. **`contracts/turbos_omm/sources/pool/rewards.move`**

```
if (!pool_rewards.reward_states.contains(&reward_type_name)) {
    let now0 = clock.timestamp_ms() / 1000;
    pool_rewards
        .reward_states
        .insert(
            reward_type_name,
            RewardState {
                emission_per_sec: 0,
                last_update_ts: now0,
                reward_per_lp: 0,
                start_ts: now0,
                end_ts: now0,
                total_emitted: 0,
                total_claimed: 0,
                reserved: 0,
            },
        );
    pool_rewards.reward_types.push_back(reward_type_name);
};
let reward_state = pool_rewards.reward_states.get_mut(&reward_type_name);
let now = clock.timestamp_ms() / 1000;
```

```
// Update reward schedule parameters
reward_state.last_update_ts = now;
reward_state.start_ts = start_ts;
reward_state.end_ts = end_ts;
reward_state.emission_per_sec = emission_per_sec;
```

This issue may lead to incorrect configurations and subsequently cause errors in reward distribution calculations.

Suggestion:

It is recommended to ensure `start_ts` is greater than current block time.

Resolution:

The team adopted our advice and fixed this issue by adding validation logic in `schedule_reward` function to ensure `start_ts` is greater than current block time, which can be found at [f781f8668286e57c71e0a302d7dc43a620293b65](#).

REW-24 Inconsistent State Updates During Reward Injection May Lead to Incorrect User Earnings Calculations

Severity: Medium

Status: Acknowledged

Code Location:

contracts/turbos_omm/sources/pool/rewards.move

Descriptions:

The `reward` contract allows administrators to inject funds into the reward pool using the `fund` function. These funds are subsequently distributed to users linearly over time based on the current inventory via the `schedule_reward` function. When calculating and recording rewards, user reward data is stored in the `Staker` struct. A single pool can support multiple reward tokens, and users are eligible to receive various staking reward tokens.

User staking token rewards must be updated by the `settle_user_one_impl` function, which can only be invoked by staking logic functions—such as `stake`, `unstake`, and `claim`—or by the `user_rewards` function, which checks the current reward status. However, when an administrator adds a new reward token, the reward information for existing users is not updated.

This creates an issue: if users are not promptly notified when a new reward token is added and do not promptly call `user_rewards` or other functions that update the reward state, the newly added reward token will not be recorded in the user's `Staker` struct. As a result, rewards that should be allocated to the user may be overlooked.

Suggestion:

It is recommended that the `fund` function should be updated to refresh the reward token status for all participating staking users when called, thereby preventing any rewards from being missed.

Resolution:

The team acknowledged this issue and state that "The `fund` function is solely used to supplement rewards and does not update emission plans. It typically executes earlier than the `schedule_reward` function. Additionally, a fund check has been implemented within `schedule_reward`, as referenced in line 295 of `reward.move`.", so no modifications will be made to the current version

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

